

[Home](#) > [My Courses](#) > [Risk Management](#) >[M8: Dynamic Bayesian Networks and Coherent Risk Management](#) > Lesson Notes

Lesson Notes

MODULE 8 | LESSON 4

BAYESIAN NETWORKS AT WORK: OIL PRICE PREDICTION

Reading Time	30 minutes
Prior Knowledge	Bayesian networks, Markov property
Keywords	Regime, Emissions, Baum-Welch algorithm, Viterbi Algorithm, Forward-Backward algorithm, Hill Climbing algorithm, Hidden Markov models (HMM)

In the last lesson, we used Bayesian networks with macroeconomic and microeconomic data, as well as expert knowledge, to estimate a bank's probability of default. In this lesson, in much the same vein and using mostly the same techniques and types of data, the task is oil price prediction for strategic trading as well as risk management. We also reintroduce Hidden Markov Models to supplement the Bayesian network model.

1. Regime Change

In this Risk Management course, we started our learning journey into Bayesian networks with a simple energy stock example; now, we finish with a reading that seeks to understand the structure of the oil price market and predict oil price movements based on dozens of variables, again with the help of Bayesian networks.

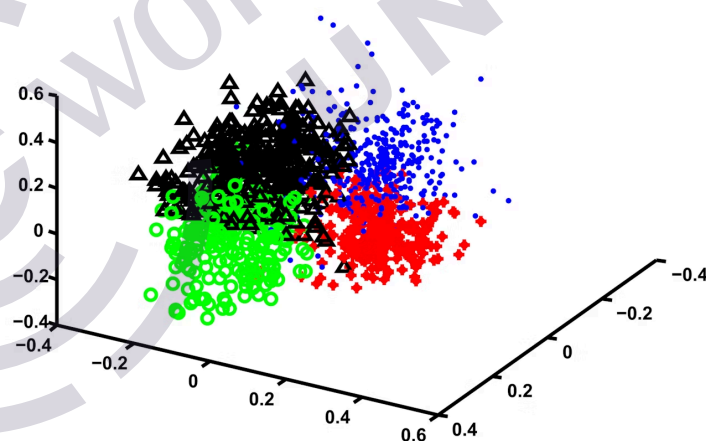
To enhance his predictions, Alvi assumes, and tries to identify, regimes in the data. Regimes can be understood broadly as intervals (not necessarily of regular length, by any means) in which the time-series variables behave in a way that is self-consistent for that interval. This variable-level self-consistency across multiple variables might pertain to the direction of a trend, the lack of a trend, high volatility or low volatility (or no volatility), excessive noise or jumpiness in the data or a noteworthy lack thereof. There are in fact signs or symptoms of a given regime, but the regime cannot be observed directly. Rather, it must be inferred by the signs or symptoms. In order to identify the regimes that crude oil price time series exhibits, Alvi uses Hidden Markov Models (HMM).

2. Hidden States

Alvi defines HMM as “as a generative probabilistic graphical model, in which a sequence of observable symbols is generated by a sequence of internal, hidden states, which are not observed directly” (22), which we can unpack some in light of the previous paragraph. HMM are generative in that they help us investigate the stochastic process that generates our data. They are graphical because, like Bayesian networks, graph structures help us visualize the relationships between variables. The observable symbols could be the upward, downward, or sideways trend; the increased or decreased volatility; etc.—because we can observe these phenomena directly. However, the cause of these changes is invisible, or hidden, to us. After trending downward for some time, the oil price might tick up a bit: Does this mean the variable has changed internal states and entered a new regime? It might, but it might not, and for this reason, the model is still probabilistic. We can only make probabilistic assertions about whether the state has changed since we cannot observe the state directly. We can only observe the phenomena that accompany each state, which are termed emissions. (Therefore, do not confuse the emissions mentioned in this oil-price research in the context of HMM with the greenhouse gas emissions that contribute to the climate risk we modeled in Module 6.)

Visser and Speekenbrink note that “the main interest is in characterizing such differences in observable responses, identifying states as meaningful clusters of co-occurring responses learned purely from the data” (2). Where we discussed “multiple variables” above, the idea of clusters better captures how the data are grouped and labeled according to some definition of similarity and literally along lines that might arguably be called arbitrary in that they are so dependent on the clustering technique, as you see in figure 1. Recall the Silhouette score and the Optimal Number of Clusters (ONC) algorithm that we used in Portfolio Management to help ensure we got the clustering right in the face of random algorithm initialization.

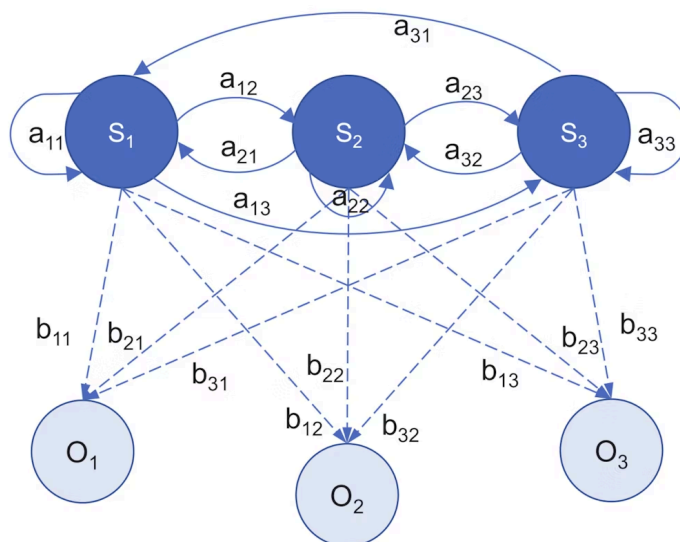
Figure 1: Data Clusters Based on Similarity



Source: Ghassempour, Sheena, et al. "Clustering Multivariate Time Series Using Hidden Markov Models." *International Journal of Environmental Research and Public Health*, vol. 11, no. 3, 2014, pp. 2741–2763, <https://www.mdpi.com/1660-4601/11/3/2741/htm>.

With that said, HMMs go beyond simple clustering of similar data points: “In hidden Markov models, where the responses are time-series, an additional interest is to characterize the process by which one state (cluster) transitions to the next” (Visser and Speekenbrink 2). Figure 1 is only relevant for visualizing the co-occurring observations, but it does not do justice to the transition process, which is at least as important to an HMM study. For that, we recall that HMM is graphical and refer the reader to figure 2.

Figure 2: A Hidden Markov Model with Three Hidden States and Three Observation States



Source: Zhang, Mingchi, et al. "A Hybrid Hidden Markov Model for Pipeline Leakage Detection." *Applied Sciences*, vol. 11, no. 7, 2021, p. 3138, <https://www.mdpi.com/2076-3417/11/7/3138/htm#>.

In the example above, we have three hidden states (in dark blue) which the techniques of HMM will help us distinguish and discern, based on the state of the observable variable (in light blue). In this example, each hidden state may emit any of the three observable states with a probability, b_{ij} , meaning it is not necessarily the case that since we observe O_i , the hidden state is necessarily some S_k ; in fact, it is quite the contrary.

3. Learning with Algorithms

More to Visser and Speekenbrink's point about the transition process, we note the probabilities represented by a_{ij} , where as you can see in figure 2 "i" and "j" represent the states being transitioned away from and to, respectively. Perhaps the best way to represent the full set of these probabilities is as a transition probability matrix A, with which you are no doubt already familiar from your previous study of Markov models. By the same token, the emission probability matrix B collects and clearly represents the values b_{ij} . Of course, both matrices must first be learned before they can be displayed. As Alvi notes, the forward-backward algorithm helps with the emissions probabilities and the Viterbi algorithm helps with the transition probabilities, while the Baum-Welch algorithm is used to tie them together by maximizing the joint likelihood (24).

To demystify these algorithms, we provide them in figures 3–5. You are not expected to learn each step of these three algorithms by heart, but you should appreciate their inputs and outputs, as well as the gist of the optimization mechanism. Note that these versions of the algorithms were adapted to a network ("hack") attack example where the emissions consist of alerts and the hidden states that we are really interested in are the attempted attacks ("attack intentions").

Figure 3: The Forward Algorithm

```

Forward_Algorithm( $\lambda, O$ ):
Input: (1) alert sequence  $O = \{alert_1, alert_2, \dots, alert_T\}$ ;
      (2) hidden Markov model (HMM)  $\lambda$ .
Output: the probability  $p(O | \lambda)$  generated by alert sequence  $O = \{alert_1, alert_2, \dots, alert_T\}$  of hidden Markov model.
Begin:
  (1)  $\forall int_{t_i} \in \lambda, 1 \leq i \leq N$ .
      //  $N$  is the number of attack intentions.
      calculate the probability of  $alert_1$  generated by  $int_{t_i}; \alpha_i(i) = \pi_i b_i(alert_1)$ 
  (2) calculate the probability of alert sequence  $\{alert_1, alert_2, \dots, alert_T\}$  and  $q_{t+1} = int_{t_i}$ .
      (a) at time  $t$ , calculate the probability of alert sequence  $\{alert_1, alert_2, \dots, alert_T\}$  and  $q_t = int_{t_i}; \alpha_i(j)$ .
      (b) at time  $t + 1$ , calculate the probability of intent sequence  $\{alert_1, alert_2, \dots, alert_T\}$  generated by hidden Markov
      model (HMM):  $\lambda$  and
       $q_t = int_{t_i}; \alpha_{t+1}(j) = \left[ \sum_{i=1}^N \alpha_t(i) a_{ij} \right] b_j(alert_{t+1})$  where  $1 \leq t \leq T - 1; 1 \leq j \leq N$ .
  (3) calculate the probability of the intent sequence  $O = \{alert_1, alert_2, \dots, alert_T\}$  generated by hidden Markov
      model (HMM):  $\lambda$ .
       $p(O | \lambda) = \sum_{i=1}^N \alpha_T(i)$ .
  (4) Return  $p(O | \lambda)$ .
End;
```

Source: Zhang, Yangxue, et al. "The Application of Baum-Welch Algorithm in Multistep Attack." *The Scientific World Journal*, 2014, <https://europepmc.org/article/med/24991642>.

Figure 4: The Viterbi Algorithm

```

Viterbi_Algorithm( $\lambda, O$ ):
Input: alert sequence  $O = \{alert_1, alert_2, \dots, alert_T\}$ ;
Output: (1) intent sequence:  $Q = \{int_{t_1}, int_{t_2}, \dots, int_{t_T}\}$ .
      (2) the completed intent sequence and the next likely intent.
Begin:
  for  $i = 1$  to HMM_m
    // HMM_m is the number of hidden Markov model(s)
    {
      Prob = Forward_Algorithm(hmm_i, O);
      // calculate the probability of alert sequence generated by each hidden Markov
      // model(s)
    }
  Most_likely_multi_step_attack_intention = maximum(Prob);
  Q = Viterbi_Algorithm(hmm_i', O);
  // Q is the completed intent sequence
  // hmm_i' is the maximum(Prob) of hmm_i
  Q' = S - Q // the next likely intent
  // S is the intent sequence of hmm_i'
End;
```

Source: Zhang, Yangxue, et al. "The Application of Baum-Welch Algorithm in Multistep Attack." *The Scientific World Journal*, 2014, <https://europepmc.org/article/med/24991642>.

Figure 5: The Baum-Welch Algorithm

```

Input: alert sequence.
       $O = \{o_1, o_2, \dots, o_T\}$ ;
Output: the parameters of hidden Markov model.
       $\lambda^{(n+1)} = (A^{(n+1)}, B^{(n+1)}, \pi^{(n+1)})$ .
Step 1. Initialization.
      for  $n = 0$ , select  $a_{ij}^{(0)}, b_j(k)^{(0)}, \pi_i^{(0)}$ , we can obtain the initial model  $\lambda^{(0)} = (A^{(0)}, B^{(0)}, \pi^{(0)})$ .
Step 2. Iterative calculation.
      for  $n = 1, 2, \dots$ ,
       $a_{ij}^{(n+1)} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$ ;
       $b_j(k)^{(n+1)} = \frac{\sum_{t=1, o_t=v_k}^T \gamma_t(j)}{\sum_{t=1}^T \gamma_t(j)}$ ;
       $\pi_i^{(n+1)} = \gamma_1(i)$ .
      where  $\gamma_t(i) = \frac{\alpha_t(i) \beta_t(i)}{p(O | \lambda)} = \frac{\alpha_t(i) \beta_t(i)}{\sum_{j=1}^N \alpha_t(j) \beta_t(j) \sum_{k=1}^M b_k(j)}$ ;
       $\xi_t(i, j) = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{p(O | \lambda)} = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{\sum_{i=1}^N \sum_{j=1}^N \alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}$ .
Step 3. Termination. We can obtain the parameters of hidden Markov model.
       $\lambda^{(n+1)} = (A^{(n+1)}, B^{(n+1)}, \pi^{(n+1)})$ .
```

Source: Zhang, Yangxue, et al. "The Application of Baum-Welch Algorithm in Multistep Attack." *The Scientific World Journal*, 2014, <https://europepmc.org/article/med/24991642>.

Setting aside this HMM material, most of the theory in this paper should be a review of what you have learned in this and the previous three modules, as you are practically an expert on Bayesian networks. It is hoped that you find the paper enlightening and suggestive with respect to the kinds of predictions and risk management decisions of which you are already capable given your theoretical and applied (read: Python) knowledge.

4. Conclusion

In this lesson, most of the material from this and the previous three modules comes together in a straightforward trading and risk management application to oil markets. In addition to reviewing and consolidating important and relevant information and concepts about Bayesian networks, it represents an example of what students in this program could accomplish in their capstone projects.

References

- Alvi, Danish. "Application of Probabilistic Graphical Models in Forecasting Crude Oil Price." 2018. University College London, Bachelor's dissertation. *arXiv*. <https://arxiv.org/abs/1804.10869>.
- Ghassempour, Sheena, et al. "Clustering Multivariate Time Series Using Hidden Markov Models." *International Journal of Environmental Research and Public Health*, vol. 11, no. 3, 2014, pp. 2741–2763, <https://www.mdpi.com/1660-4601/11/3/2741/htm>.
- Visser, Ingmar, and Maarten Speekenbrink. *Mixture and Hidden Markov Models with R*. Springer, <https://doi.org/10.1007/978-3-031-01440-6>.
- Zhang, Mingchi, et al. "A Hybrid Hidden Markov Model for Pipeline Leakage Detection." *Applied Sciences*, vol. 11, no. 7, 2021, p. 3138, <https://www.mdpi.com/2076-3417/11/7/3138/htm#>.
- Zhang, Yangxue, et al. "The Application of Baum-Welch Algorithm in Multistep Attack." *The Scientific World Journal*, 2014, <https://europemc.org/article/med/24991642>.